INDEX

NAME Adithya S SUBJECT Computer Network Obs

STD. DIV. ROLL NO. 240701019 SCHOOL

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1)	1-6	Study of Linux and Windows Commands	27-7-26	28/8/26 (8)
2)	7-8	Studies of different types of Network Cables	3-8-26	(7.5/10) (8)
3)	9-11	Experiments on CISCO PACKET TRACER (Simulation Table)	3-8-26	10/9/26 (10)
4)	03/08	Spitting up a Local Area Network [LAN]	03/08	3/8 (9)
5)	18/08	Packet Capture Tool: Wireshark	18/08	18/8 (9)
6)	07/09	Error Correction at Data Link Layer	07/09	7/9 (10)
7)	08/09	Flow Control at Data Link Layer using Sliding Window Protocol	08/09	8/9 (10)
8)	08/09	NMAP to Discover the Live Host using Nmap scans on the TryHackme Platform	08/09	8/9 (10)
9)	22/09	Implementation of Subnetting in CISCO PACKET TRACER Simulator	22/09	22/9 (9)
12)	29/09	End-To-End Communication at Data Link Layer	29/09	29/9 (9)
13)	30/9	Implementing Own PING Program	30/09	30/9 (9)
14)	01/10	Packet Sniffing using RAW Source	01/10	1/10 (9)

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
10)	8/10	Internetworking with Routers in Cisco Packet Tracer.	8/10	9 8/10
11)	8/10	Routing Information Protocol	8/10	10 8/10

Completed
8/10/26

Exp-1

Practical - I

* **Aim:**

Study of Various Network Commands used in Linux and Windows.

* **Basic Networking Commands:**

- arp -a : Displays the IP address and MAC Address connected to your network
- hostname : Displays the name of Computer.
- ipconfig /all : Shows complete network info like IP address, MAC address, DNS, etc
- nbtstat -a : Shows NETBIOS information of a remote Computer.
- netstat : Display active network connections, listening ports, routing table.
- netstat -r : Displays the Routing Table.
- nslookup : Finds the IP Address of a website or gets DNS information.
- pathping : Shows the route packets take to the destination.
- ping : Checks whether another computer is reachable and measures response time.
- ping localhost : Test whether the computer's stack is working or not.

- ping IP Address: Test connectivity to a specific address
- ping Domain Name: Test connectivity to a website using domain name
- route: Displays or modifies the IP routing table

* Basic Linux Networking Commands:

- ip: It is a networking Linux command used to view and manage IP address, networking Interface and routes.
- ifconfig: It displays and configures network interface.

  ifconfig is older version which is replaced by IP.
- mtr:

  mtr [My Trace Route] combines ping and traceroute. It shows the path packet takes and checks packet loss at each router.
- tcpdump:

  It captures and displays network packets for troubleshooting and network analysis.

Installation Command :- dnf install -y tcpdump

before capture :- tcpdump -D

for traffic on etho :- tcpdump -i etho

Capture traffic to and from a host :

tcpdump -i etho -c 10 host 8.8.8.8

For traffic from 8.8.8.8 :

tcpdump -i etho src host 8.8.8.8

To and From network :

tcpdump -i etho net 10.1.0.0 mask 255.255.255.0

(or)

tcpdump -i etho net 10.1.0.0/24

To and From port numbers :

tcpdump -i etho port 53

host :

tcpdump -i etho 8.8.8.8 and port 53

HTTPS :

tcpdump -i etho -c 10 host www.google.com and port 443

• 8. **ping**:

Tool to verify IP-Level Connectivity to another TCP/IP by sending ICMP Echo request message receipt of echo is displayed. Used to troubleshoot connectivity, reachability and name resolution.

Ex: ping google.com

⇒ ping google.com (216.58.206.174) 56(84 bytes of data.

⇒ 64 bytes from soft 02527-in-f14.1e100.net

Stopped by CTRL+C

If needed to ping 10 times:-

ping -c 10 google.com

Configuring Ethernet connection by using nmcli.

Can manage connections setting in CLI using nmcli utility.

Procedure:

1. List network manages connection profiles

⇒ nmcli connection show

2. nmcli connection add con-name <connection-name> ifname <device-name> type ethernet

3. Display current settings :

nmcli connection show

4. Configure IPv4 settings :-

edit DHCP by :

nmcli connection modify "Wired connection 1"
ipv4.method auto

Osk ip if method → auto

Set Static ipv4 address etc :

nmcli connection modify "Wired Connection 1"
ipv4.method manual ipv4.addresses 192.0.2.1/24
ipv4.gateway 192.0.2.254 ipv4.dns 192.0.2.200
ipv4.dns-search examble.com.

5. Configure IPv6 Settings :

DHCP :

Command Similar to IPv 4

Verification :

1. Display IP Setting of NIC :-

ip address show enp1s0

2. Display IPv4 default :

ip route show default

3. Display IPV6 default :

ip -6 route show default

4. Display DNS :

cat /etc/resolv.conf

5. Use ping to verify that host can send packets to other hosts

ping <host_name_or_IPaddress>

26/12/26

Result

Hence the study of various Linux and Windows commands were executed successfully

Exp: 2

Studies of different types of Network Cables

Aim:

a) Understand different types of network cable

Different type of cables and in networking are:

1. unshielded Twisted Pair (UTP) cable
2. Shielded Twisted Pair (STP) Cable
3. Coaxial Cable
4. Fibre Optic Cable

Cable Type	Category	Maximum Data	Advantages / Disadvantages	Application
UTP	Category 3	10 bps	Advantages: • cheaper in • carry to install as they have a smaller overall diameter Dis-Advantages: • More prone to (EMI) Electromagnetic interference	10 Blue - 7 Ethernet Fast, Ethernet Gigabit Ethernet
	Category 5	Up to 100 rbps		
	Category 5e	1 Gbps		Fast Ethernet, Gigabit Ethernet
STP	Category 6 6a	10 Gbps	Advantages • Shielded • Faster than UTP • Less acceptible to noise and interference Disadvantages: • Expensive • Greater Installation Effort	
	Category 7	10 Gbps		

Cable Type	Category	Maximum Data Transmission	Advantages / Disadvantages	Applications
Co-axial	RG-6 RG-59 RG-11	10-100 Mbps	• Greater Installation Effort Advantages: • High bandwidth • Immune to interference • versatile • Low Loss bandwidth Disadvantages: • Limited	Speed of Signal is 500 m Television network High Speed Internet Connection
Fibre Optics Cable	Single Mode	100 Gbps	Advantages: • High Speed • High bandwidth • High security • Long distance Disadvantages • Expensive	Maximum distance of fibre optic cable is around 100m
	Multi Mode			

Result.

Thus the different types of cables were studied and understood successfully

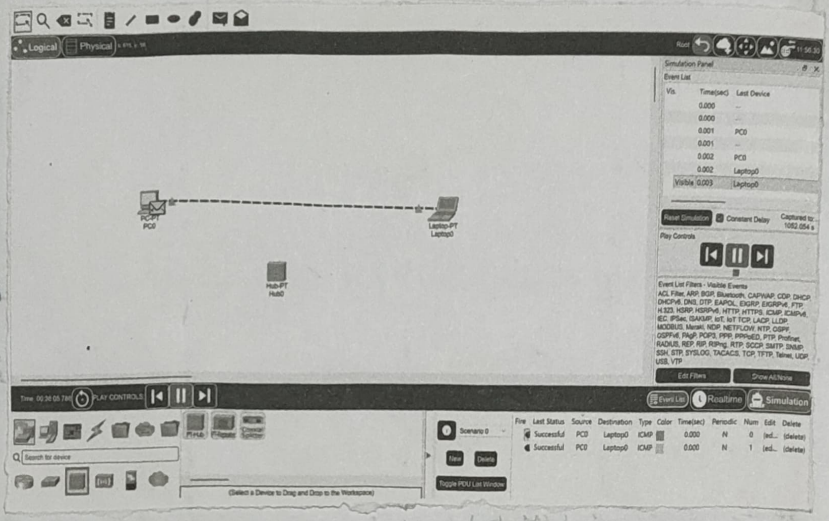
Exp-3

Experiments on CISCO PACKET TRACER (Simulation Table)

Aim :

To understand its environment and behaviour of network devices of CISCO packet tracer simulator.

1) Peer to Peer Connection (P2P) :-

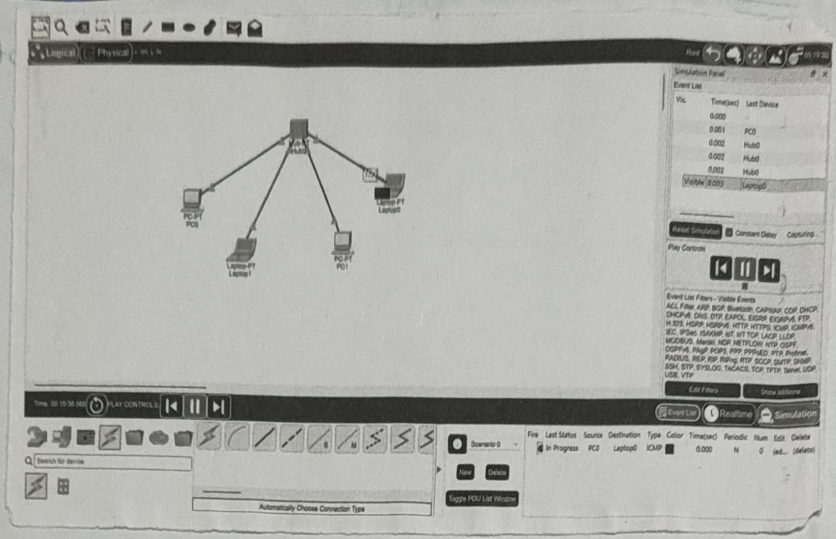


* Cross-Over cables were used to connect the PC and the laptop.

* Only a cross-over cable can enables communication directly between same type of End-Devices.

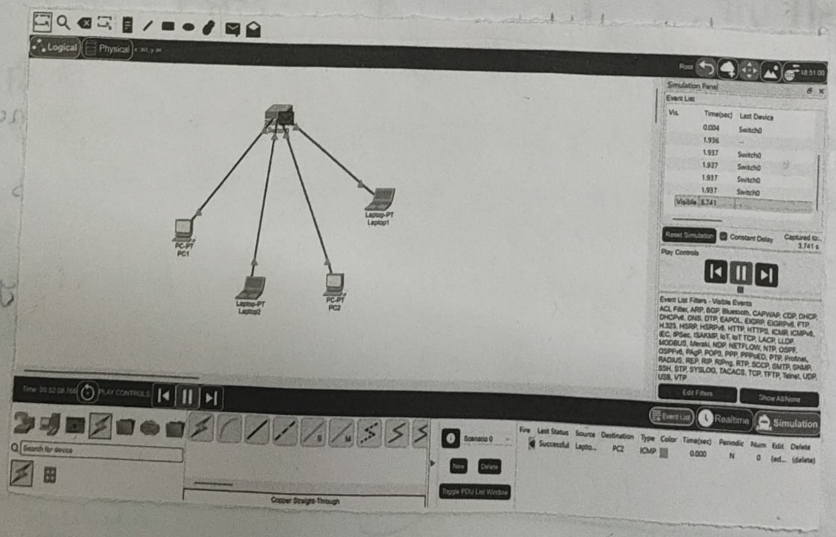
* Usage of such cables have been replaced by switches, hubs and Routers.

2) Hub Connection: -



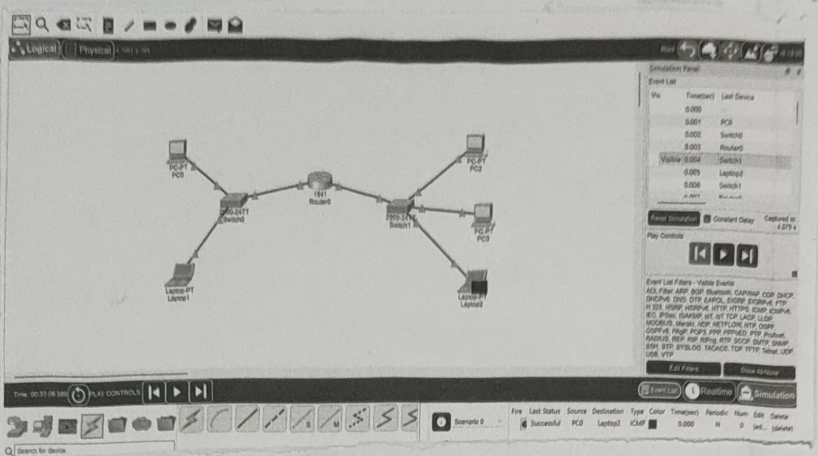
* A hub connects multiple devices in a network by blindly broadcasting all incoming data signals to every connected devices.

3) Switch Connection



* Switch connects multiple devices within single LAN and intelligently forwards data packets directly to specific destination using MAC Addresses

4) Router Connection :



* A router connects differend subnets or distinct networks together, directing data traffic between them using IP addresses and default gateways.

Result :

Hence the environment of CISCO PACKET TRACER and behaviour of all the network devices in the software were studied, simulated and verified successfully.

10/8/20

Exp No: 4

03/08/2024

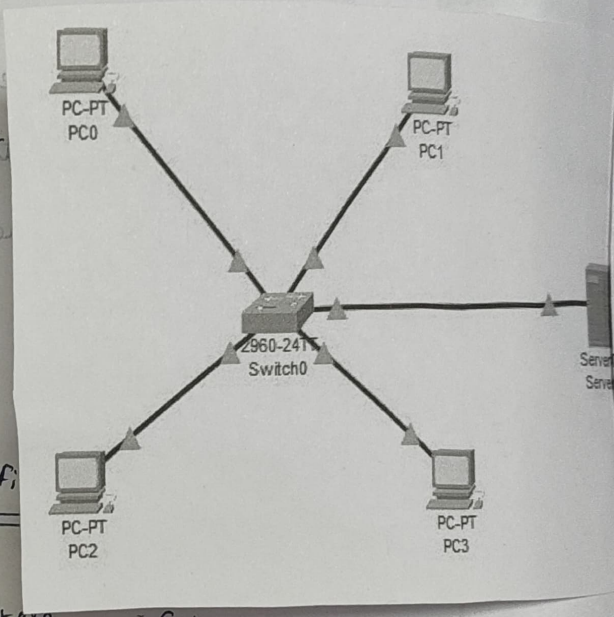
Splitting Up A Local Area Network [LAN]

Aim:

To design, set up and configure a local area Network [LAN] connecting four computers through a network switch and demonstrate resource / data sharing across the network.

Equipment Required:

Equipment
Personal Computer
Network Switch
Ethernet Cables [cat5e / cat6]



IP address Confi

Device	IP Address	Subnet Mask	Connected to
PC0	10.1.1.1	255.0.0.0	Switch Port Fa 0/
PC1	10.1.1.2	255.0.0.0	Switch Port Fa 0/2
PC2	10.1.1.3	255.0.0.0	Switch Port Fa 0/3
PC3	10.1.1.4	255.0.0.0	Switch Port Fa 0/4
Switch	10.1.1.5	255.0.0.0	VLAN 1, Interface

Implementation is CISCO Packet Tracer

1) Place the Devices:

→ Open Cisco Packet Tracer

→ From the device category Panel, Select End Devices and drag 4 PC onto the workspace

→ Select Switches and Drag Switch

2) Cable the Switch / Device:

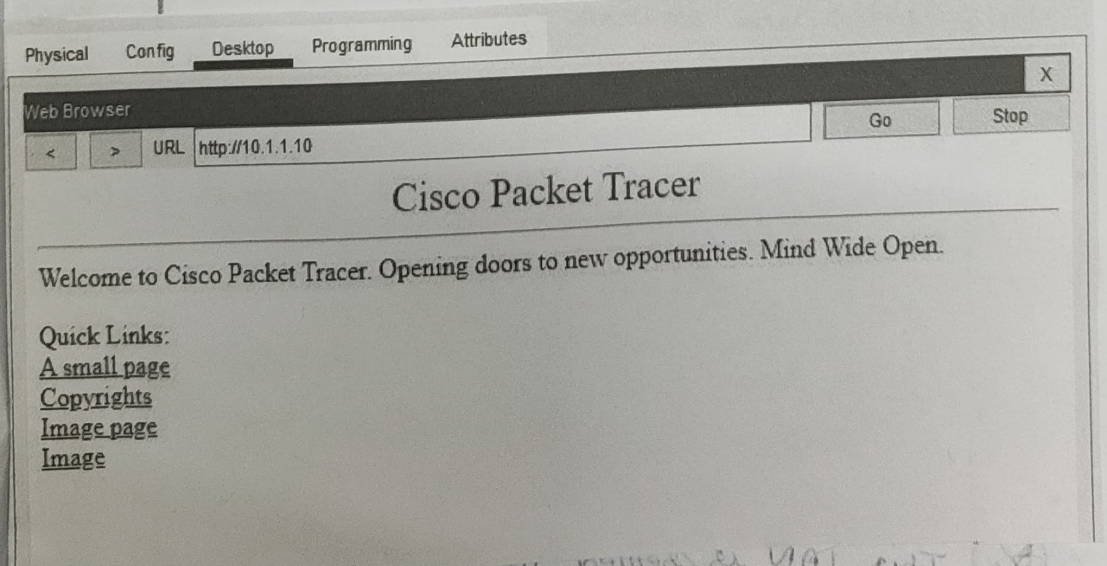
→ Click Connections → Select Copper straight - through Cables

→ Click each PC, then click the switch and select an available Fast Ethernet Port to complete each connection.

→ Repeat for all four PC's Ports [Fa 0/1 to Fa 0/4]
Green Link Lights confirm a successful Physical Connection

3) Assign IP Addresses to PC's:

→ Click a PC - Desktop Tab - IP Configuration



No Shutdown

exit

5) Verify Connectivity using Ping

→ Click a PC → Desktop Tab → and Prompt

→ Use Ping to test connectivity

→ Successful Message!

6) Demonstrate Data Sharing [Server Simulation]

→ Add a Server device [End Device → Server → PT] and connect it to Switch.

→ Assign it an IP Address

→ Open the Server's Server Tab & enables the HTTP

→ From another PC, open Web Browser and enter the Server's IP address to load the Page

Confirmation. Successful data Transfer across LAN.

Result:

This LAN is settled up and connection is verified Successfully

Ex: 05 Experiments on Packet Tr Capture Tool WireShark

18/8/26

Aim:

Experiment on Packet Capture tool WireShark

WireShark:

WireShark is a network analysis tool formerly known as Ethernal, Captures packet in real-time and display them in real time and display them in human readable format. WireShark includes filter's, color coding and other features that let you dig deep into network traffic and inspect individual packets.

Capturing packets:

⇒ After downloading & Installing wireshark, launch it and download it.

⇒ Select local area connection in wireshark

⇒ Go to capture Option.

⇒ Select stop capture automatically after 100 packets.

⇒ Save the packets.

Filtering packets:

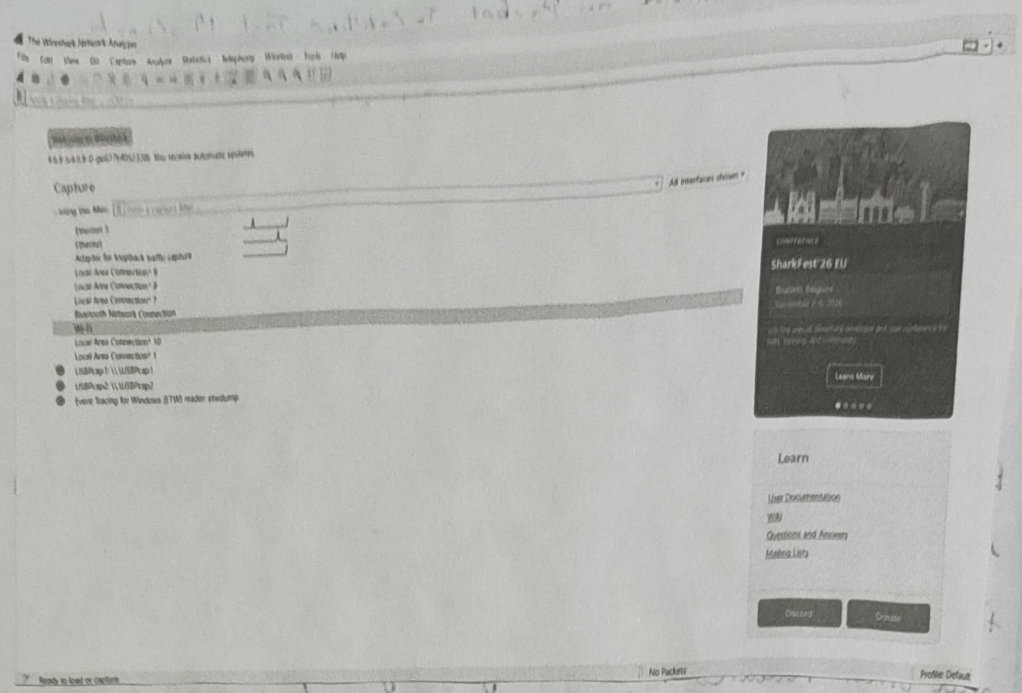
⇒ Go to capture option.

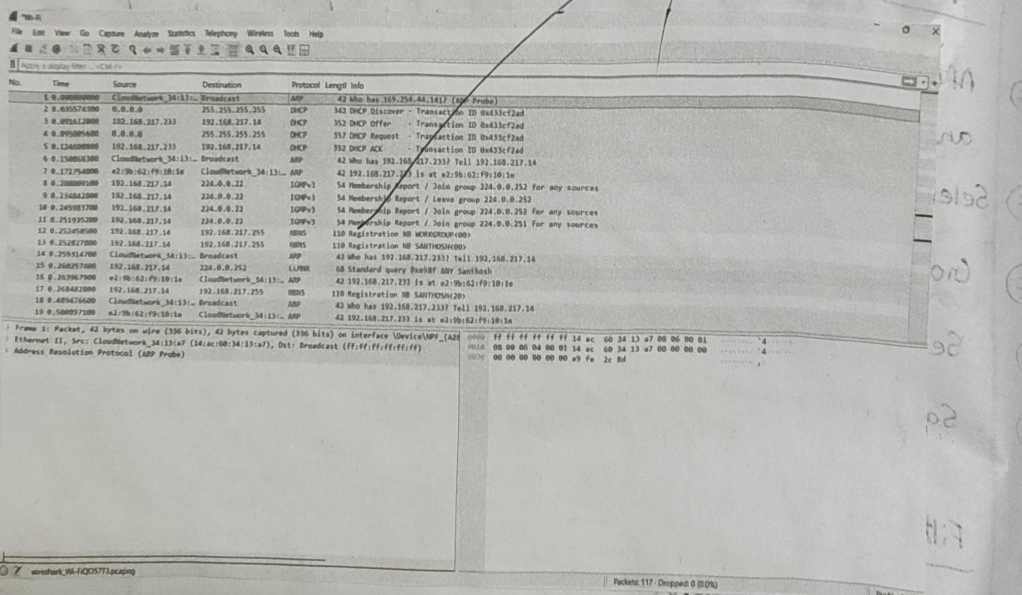
⇒ Select Stop capture automatically after 100 packets

⇒ Then click Start capture

⇒ Search ARP Req Packets in Search bar

⇒ Save the packet.





18/8/26

Result:

Capturing & filtering of packets using wireshark

Exp-6

7/9/26

Practical - 6 Hamming Code

Aim:

Write program to implement error detection and correction using hamming code concept. Make test to run input data stream & verify correction feature.

Code:

```
def calc_redundant_bits(m):
    for r in range(m+1):
        if(2**r) >= (m+r+1):
            return r

def main():
    data = input("Enter data bits : ").strip()
    m = len(data)
    r = calc_redundant_bits(m)
    n = m + r
    print(f"Number of r needed : {r}")
    print(f"Total frame length: {n}")
    code = ['0'] * (n+1)
    j=0
    for i in range(1,n+1):
        if(i & (i-1)) != 0:
            code[i] = data[(j+1)]
            j+=1
    for i in range(r):
        parity_pos = 2**i
        val=0
        for pos in range(1,n+1):
            if(pos & parity_pos)!=0 and pos != parity_pos:
                if code[pos]=='1':
```

```
code[parity-pos] = str(val)

transmitted_str = "".join(code[1:][::-1])

print(f"Data to be transmitted : {transmitted_str}")

change = input("\n transmitted bit change").strip().lower()

if change == "yes":

    received_str = input("Enter corrupted data")

    if (int(received_str)) != n:

        print(f"Error : Received data & be exactly {n} bits avg !")

        return

    received_code = ['0'] + list(received_str[::-1])

else:

    received_code = list(code)

error_position = 0

for i in range(9):

    parity_pos = 2**i

    val = 0

    for pos in range(1, n+1):

        if (pos & parity_pos) != 0:

            val ^= 0

if error == 0:

    print("No error detected")

else:

    print("error detected at position", error)

    code[error] = 1 - code[error]

    print("error detected")
```

```
corrected_binary = ""
for i in range(1, n+1):
    if i & (i-1) != 0:
        corrected_binary += str(code[i])
corrected_binary = corrected_binary[:k]
corrected_text = binary_to_text(corrected_binary)
print("corrected_text:", corrected_text)
```

Result:

Thus, error detection and correction is done using Hamming Code.

Expt No: 7

8/9/26

Flow Control At data Link Layer using Sliding window Protocol

Aim:

To write a program to implement flow control at data Link Layer using Sliding window Protocol, simulate the flow of frames from one node to another

* Create a Sender Program with following features:

(i) Input window Size from the user

(ii) Input a text message from the user.

(iii) Consider one character per frame,

(iv) Create a frame with following fields

(v) Send the frames [print output on session & save it in a file called sender-buffer]

(vi) Wait for acknowledgement from receiver

(vii) Read a file called Receiver Buffer.

(viii) Check Ack filled the acknowledgement number.

* Create a receiver file with following features:

(i) Read a file - called sender-buffer

(ii) Check frame number

(iii) If frame no: is as expected, write appropriate acknowledgement number in

receiver_buffer else write to NACK number in receiver buffer.

=) Sender Code :

```
import os
import time
Base_dir = os.path.dirname(os.path.abspath(__)
print("Sender buffer:", sender_file)
print("Receiver buffer :", receiver_file)
window_size = int(input("Enter Window size:"))
message = input("Enter text message:")
if window_size <=0:
    print("window size must be greater than 0")
    false systemexit;
frames[]
while start < total_frames:
    end = min(start + window_size, total_frams
    print("\n =====")
    print("Sending frames", start+1, "to", end)
    print("\n =====")
elif ack_type = "NAK"
    print("NACK received")
    print("Retransmitting from frames:", ack_no)
else:
    print("Unknown response : Retrying current window")
print("\n=====")
print("Total frames", total_frames)
```

=) Receiver Program:

```
import time
import OS
expected = 0
while true:
    if not OS.path.exists("Sender_buffer_txt"):
        time.sleep(1)
        continue
    with open("send_buffer_txt") as f:
        frames = f.readlines()

    if not frames:
        time.sleep(1)
        continue
    error = false
    for for line in frames:
        n, data = line.strip().split(",")
        n = int(n)
        print("Received", n, data)
        if n != expected:
            with open("Receiver_buffer_txt", "w")
                f.write("NACK:" + str(expected))
            error = true
            break
    expected += 1
if not error:
    with open("Receiver_buffer_text", "w") as f:
        f.write("ACK" + str(expected))
```

.open ("Sender_buffer.txt", "w").close()

time.sleep(1)

Sender code input:

window size = 3

message = HELLO

Sender Code Output:

Frame 0 : H

1 : E

2 : L

3 : L

4 : O

Receiver Code Input:

Sender_buffer_text

0 : H

1 : E

2 : L

3 : L

4 : O

Receiver Code Output:

Received:

0 : H

1 : E

2 : L

3 : L

4 : O

Sender code output:

Window size = 3

message = HELLO

Sender Code Output:

frame 0 : H

1 : E

2 : L

3 : L

4 : O

Receiver Code Output:

Sender buffer - test

0 : H

1 : E

2 : L

3 : L

4 : O

Receiver Code Output:

Received:

0 : H

1 : E

2 : L

3 : L

4 : O

8/9/26

Result:

The Sliding Window Protocol was successfully implemented to control the frame flow at data Link Layer.

Exp No: 8

8/9/26

NMAP to Discover the Live Host using Nmap Scans (ARP, TCP/UDP) on the Tryhackme Platform

Aim:

To discover Live hosts using nmap scans (ARP, TCP/UDP, ICMP) on the tryhackme Platform.

Procedure:

1) Setup and environment Initialization:

- Log in to TryHackme Platform and navigate to Target room: Tryhackme Nmap Room
- Deploy the Target virtual machine and establish a VPN connection using OpenVPN or Launch the browser based attach box

2) ARP Scan for local Host Discovery:

Perform an ARP Request scan to identify Live host on the Local network Segment

Command:

Sudo nmap -PR -sn target-IP <subnet>

3) ICMP Echo Ping Scan:

Send TCMP Echo Request Packets to check for TCP Echo reply from Active host.

4) Perform TCMP Timestamp Ping Scan:

Echo Request:

Command:

Sudo nmap -PP -sn <Target IP/ Subnet>

5) Send TCP address mask request Packet to Identify live systems

CMD:

Sudo nmap -PR -sn <Target-IP / Subnet>

6) Send TCP SYN Packets to specified Ports to discover Systems based on SYN ACK

CMD:

Sudo nmap -PS -sn <Target-IP / Subnet>

7) TCP Ack Packets sent to receive RST response from active host

CMD:

Sudo nmap -PR -sn <Target IP / Subnet>

8) Send empty UDP Packets to destination Ports to detect host via TCMP port unreachable response

CMD:

Sudo nmap -PU -sn <Target-IP / Subnet>

9) Use the sn flag with nmap to perform host discovery exclusively without triggering subsequent Port Scans.

CMD:

Sudo nmap -sn <Target-IP / Subnet>

~~8~~ 9

Result:

Thus, live host discovery using various nmap scanning Techniques was successfully implemented on the TryHackme Platform

Exp No: 9
22/9/26

Implementation of Subnetting in CISCO Packet Tracer Simulator

Aim

To implement classless IP Subnetting using CISCO PACKET TRACER and verify connectivity between different Subnets.

Procedure:

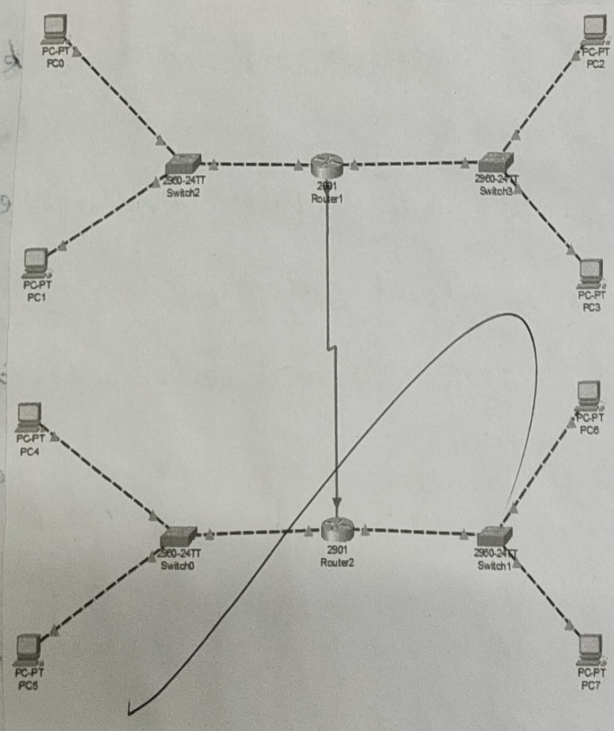
1. Open CISCO Packet Tracer and create a new network Topology.
2. Add routers, Switches and PCs to the workspace
3. Connect the devices using appropriate cables.
4. Divide the network 192.168.1.0/24 into smaller subnets using the /27 subnet mask
5. Configure the IP addresses and subnet mask for the router interfaces.
6. Configure the PC's with Suitable IP addresses, Subnet masks & default gateway.
7. Configure the Switch Ports and router interface
8. Use the Ping Command to test connectivity between devices.
9. Verify the Packets are successfully Transmitted between the connected devices.

Advantages

- Efficient use of IP address
- Reduces network Congestion
- Improves network management

IP Addressing:

Network	Subnet Mask	Router IP	PC IP-Range
192.168.1.0/27	255.255.255.224	192.168.1.1	192.168.1.11 - 15
192.168.2.0/27	255.255.255.224	192.168.2.1	192.168.2.11 - 15
192.168.3.0/27	255.255.255.224	192.168.3.1	192.168.3.11 - 15
192.168.4.0/27	255.255.255.224	192.168.4.1	192.168.4.11 - 15



Result:

The Classless IP Subnetting was successfully implemented using CISCO PACKET TRACER.

EXPT NO: 12

29/09/26

END-TO-END COMMUNICATION AT TRANSPORT LAYER

(a) Implement ECHO Client, Server using TCP Sockets

Aim:

To implement an ECHO SERVER application using TCP Sockets.

Code:

ECHO SERVER:

```
import socket
s = socket.socket()
s.bind(("localhost", 5000))
s.listen(1)
c, addr = s.accept()
msg = c.recv(1024).decode()
print("Client:", msg)
c.send(msg.encode())
c.close()
s.close()
```

ECHO CLIENT:

```
import socket
s = socket.socket()
s.connect(("localhost", 6000))
msg = input("Enter Message:")
s.send(msg.encode())
print("Echo:", s.recv(1024).decode())
s.close()
```

Input :

Enter message : Hello Server

Output :

Server

Client Connected

Received : Hello Server

Client

Echo from Server : Hello Server

Result :

The ECHO Client Server using TCP Socket was studied and executed successfully.

(b) Implement Chat Client Server using TCP/IP Sockets

Aim :

To implement a chat Clierd Server Application using TCP Socket

Code :

SERVER:

```
import socket
s= socket.socket()
s.bind(("localhost", 6000))
s.listen(1)
c, a = s.accept()
while True:
    msg = c.recv(1024).decode()
    print("Client:", msg)
    if msg == "bye"
        break
    reply = input("server:")
    c.send(reply.encode())
c.close()
```

CLIENT

```
import socket
s = socket.socket()
s.connect(("localhost:", 6000))
while TRUE True:
    msg = input ("Client:")
    s.send (msg.encode())
    if msg == "byte":
        break
    print ("Server:", s.recv(1024).decode())
s.close()
```

Input:

Client: Hello

Server: Hi

Client: Bye

Output:

Client: Hello

Server: Hi

Result:

The chat client server Applications were successfully studied and implemented using TCP Sockets.

29/9/22

EXPT NO: 13

30/09/24 Implement your Own PING Program

Aim :

To implement a Ping Program using Python to check the connectivity between source and destination host.

Algorithm :

(i) Get the Destination IP Address from users

(ii) Send the request to destination.

(iii) Wait for a response.

(iv) Calculate the response Time

(v) Display whether the host is reachable.

(vi) Repeat the Process for few request.

CODE :

```
import OS
import time
host = input ("Enter host:")
for i in range (4):
    start = time.time ()
    result = OS.system ("Ping -n 1" + host + " >nul")
    end = time.time ()
    if result == 0 :
        print ("Reply from", host, "Time =",
               round ((end - start)*1000, 2), "ms")
    else:
        print ("Request timed out")
time.sleep(1)
```

INPUT :

Enter host: 127.0.0.1

Output :

Reply from 127.0.0.1 Time = 1.23 ms

Reply from 127.0.0.1 Time = 0.98 ms

Result :

The Ping Program was Successfully studied & implemented.

EXP NO: 19

01/10/25

Packet Sniffing Using Raw Socket

Aim:

To write a Program using RAW sockets to capture and display network Packets.

Algorithm:

(i) Create a RAW Socket

(ii) Bind the Socket to the network Interface

(iii) Receive Packets from the network

(iv) Extract the IP header information.

(v) Display the Source IP, destination IP and Protocol.

(vi) Continue Capturing.

CODE:

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_RAW,
socket.IPPROTO_IP)
s.bind(("0.0.0.0", 0))
s.setsocket(socket.IPPROTO_IP, Socket
            IP_HDRINCL, 1)
print("Packet Sniffing Started...")
```

```
while True:
    data, addr = s.recv from (65535)
    print ("Packet Received from :", addr[0])
```

Input :

Network Packets

Output :

Packet Sniffing Started ...

Packet Received from : 192.168.1.15

Packet Received from : 192.168.183.14

Packet Received from : 192.120.85.14

Result

The RAW SOCKET SNIFFING PROGRAM was successfully implemented and network Packet was captured and displayed.

Exp 10

8/10/26

Internetworking with Routers in Cisco Packet Tracer

Aim :

To design and configure a Simple internetworking Using a router in Cisco Packet Tracer and fast connectivity between 2 Pc's

Procedure :

Step : 1 → Configuring Router :

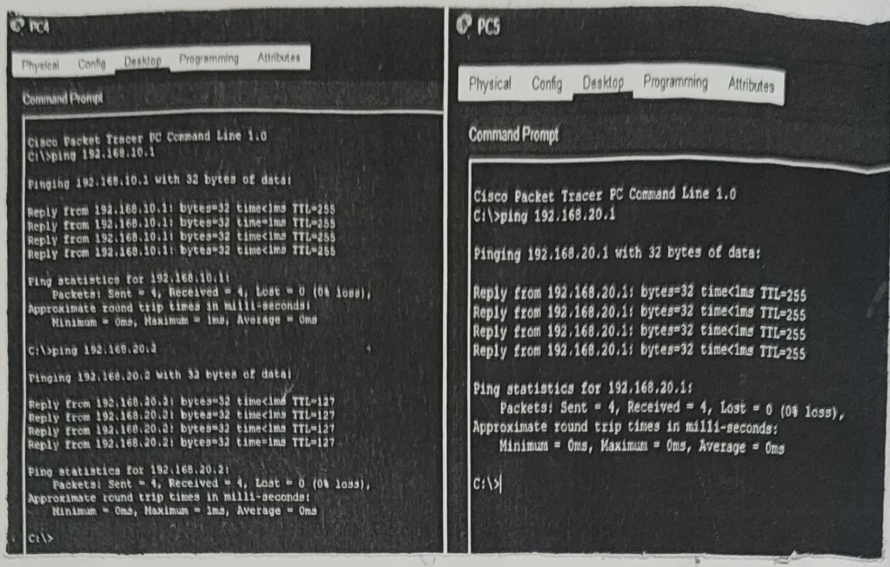
* Select the router and Open CLI
* Press enter to Start Configure Router
* Type enable to activate the previleged mode

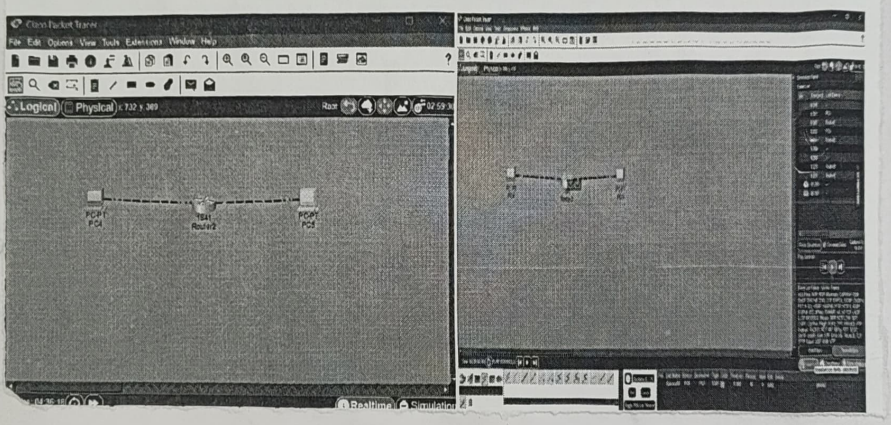
Step : 2 →

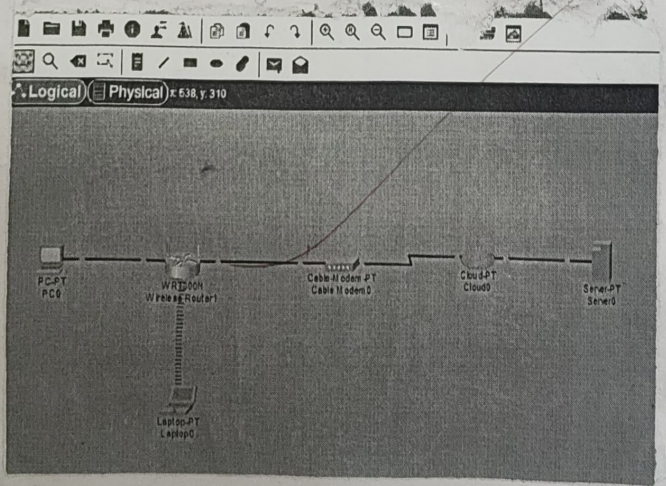
* Assign Ip address to every PC in the network
* Select the PC, Go to the desktop and Select IP configuration and assign an IP address default gateway
* Assign default gateway for both PC's

Step : 3 →

* Connet first ethernet for both pc's through Copper Straight - through Cable
* Connect first ethernet router using a Copper Straight through Cable







8/10/24

Result :

Thus, the simple internetworking with routers is successfully designed and

Exp 11

8/10/26

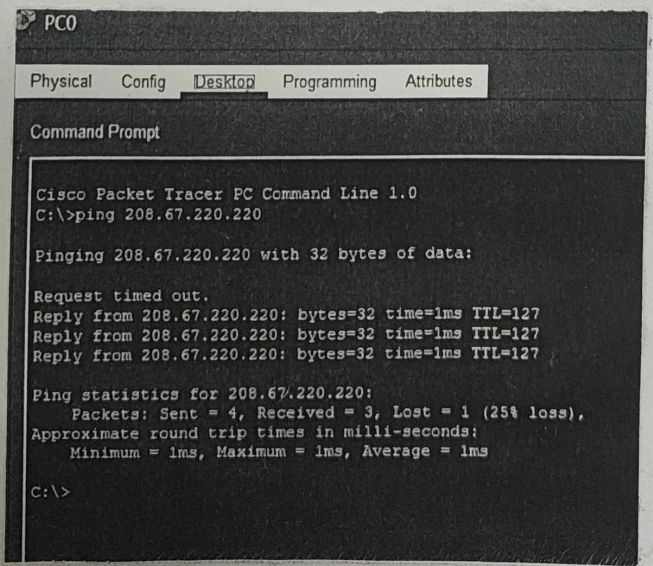
Routing Information Protocol

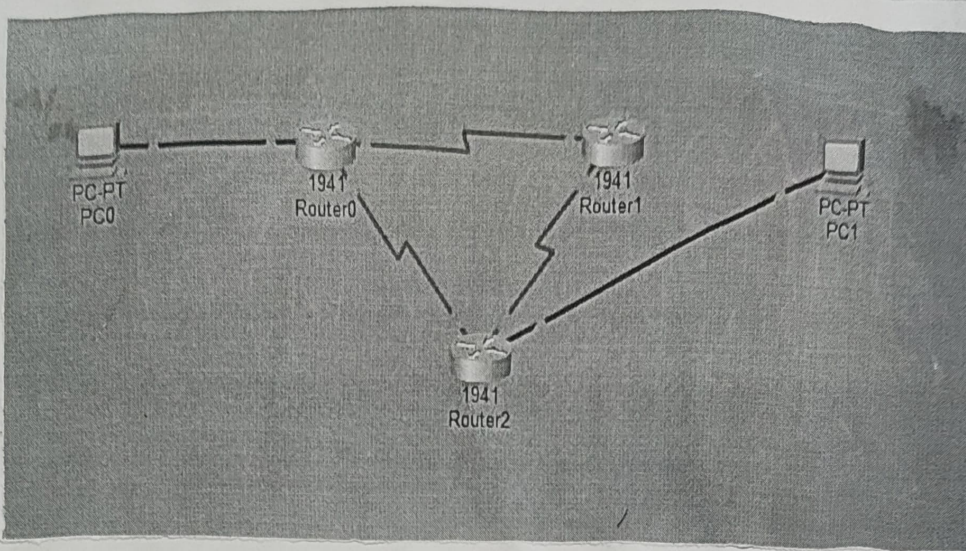
Aim

To Stimulate a Static routing Configuration Using Cisco Packet Tracer.

Procedure

* Open Cisco Packet Tracer and create the given network topology.
* Connect 3 routers and PC's Using Suitable Cables
* Configure IP Addresses for routers and PC's
* Enable all router interface Using the Shutdown.
* Configure Static routers on router of router 1 and router 2 Using IP mobile.
* Set main and backup router Using different administrative distance Value.
* Test Connectivity.
* Use tracer to verify the Routing Path





PC0

Physical | Config | Desktop | Programming | Attributes

Command Prompt

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 30.0.0.100

Pinging 30.0.0.100 with 32 bytes of data:

Request timed out.
Reply from 30.0.0.100: bytes=32 time=2ms TTL=126
Reply from 30.0.0.100: bytes=32 time=18ms TTL=126
Reply from 30.0.0.100: bytes=32 time=24ms TTL=126

Ping statistics for 30.0.0.100:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 2ms, Maximum = 24ms, Average = 14ms

C:\>tracert 30.0.0.100

Tracing route to 30.0.0.100 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2    1 ms      0 ms      0 ms      40.0.0.2
  3    3 ms      2 ms      2 ms      20.0.0.2
  4    2 ms      5 ms      0 ms      30.0.0.100

Trace complete.

C:\>
```

PC0

Physical | Config | Desktop | Programming | Attributes

Command Prompt

```
C:\>ping cisco.com

Pinging 208.67.220.220 with 32 bytes of data:

Reply from 208.67.220.220: bytes=32 time=9ms TTL=127
Reply from 208.67.220.220: bytes=32 time=1ms TTL=127
Reply from 208.67.220.220: bytes=32 time=1ms TTL=127
Reply from 208.67.220.220: bytes=32 time=2ms TTL=127

Ping statistics for 208.67.220.220:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 9ms, Average = 3ms
```

8/10/24

Result :

RIP Successfully Configure and dynamic routing between PC